UNIVERSITÀ DI TRENTO

Dipartimento di Ingegneria Industriale
Curriculum di Robotica e Meccatronica

Ottimizzazione e Parallelizzazione di Convoluzione 2D con OpenMP

Anno accademico 2025/2026

Andrea Lorenzi - Matricola 244271

1. Introduzione e Setup Sperimentale

1.1 Contesto e Obiettivo dell'Analisi

Il presente documento analizza le prestazioni e la scalabilità di un algoritmo in C++ per la convoluzione di matrici, parallelizzato mediante le direttive a memoria condivisa dello standard OpenMP. L'operazione di convoluzione, fondamentale nell'elaborazione delle immagini e nel machine learning, consiste nell'applicazione di un filtro (kernel) di dimensione fissa 4x4 su un'immagine di input (rappresentata come matrice quadrata linearizzata in memoria) di dimensione variabile da N=512 a N=4096. Essendo un problema caratterizzato da cicli fortemente annidati con complessità computazionale proporzionale a O(N^2 x K^2), si presta in modo eccellente a un'indagine sul parallelismo *data-parallel*.

1.2 Architettura Hardware

Poiché le performance in ambiente multithreading sono strettamente legate all'architettura sottostante, i benchmark sono stati condotti sul seguente nodo computazionale:

- **CPU:** Intel(R) Core(TM) i7-4700MQ. Si tratta di un processore dotato di **4 Core fisici** e **8 Thread logici**. Questa specifica è cruciale per l'interpretazione dei dati: ci si aspetta uno scaling quasi lineare fino a 4 thread, un beneficio marginale tra 4 e 8 thread (dovuto all'ottimizzazione delle pipeline del singolo core), e un degrado delle prestazioni per *oversubscription* (eccessivo context-switching) oltre gli 8 thread.

- **Memoria RAM:** 16 GB DDR3. La tecnologia DDR3, pur garantendo un quantitativo di memoria sufficiente a ospitare le matrici più grandi (es. una matrice 4096x4096 di *float* occupa circa 67 MB per l'input e altrettanti per l'output), presenta una banda passante limitata rispetto agli standard moderni. Questo costituirà un fattore determinante per l'osservazione del fenomeno del *Memory Wall* al crescere del parallelismo.

1.3 Stack Software e Compilazione

Il codice è stato compilato utilizzando **g++ (GNU Compiler Collection)**. Per garantire che l'analisi si concentrasse esclusivamente sui colli di bottiglia del parallelismo e non su inefficienze sequenziali del codice, sono stati applicati i seguenti flag:

- ***-O3***: Attiva il massimo livello di ottimizzazione sicura del compilatore, includendo il *loop unrolling*, l'inlining delle funzioni e, fattore fondamentale per i cicli di convoluzione, la vettorizzazione automatica (SIMD) dove possibile.
- ***-fopenmp:*** Abilita il supporto al multithreading tramite le librerie OpenMP, istruendo il compilatore a interpretare le direttive *#pragma* inserite nel codice.

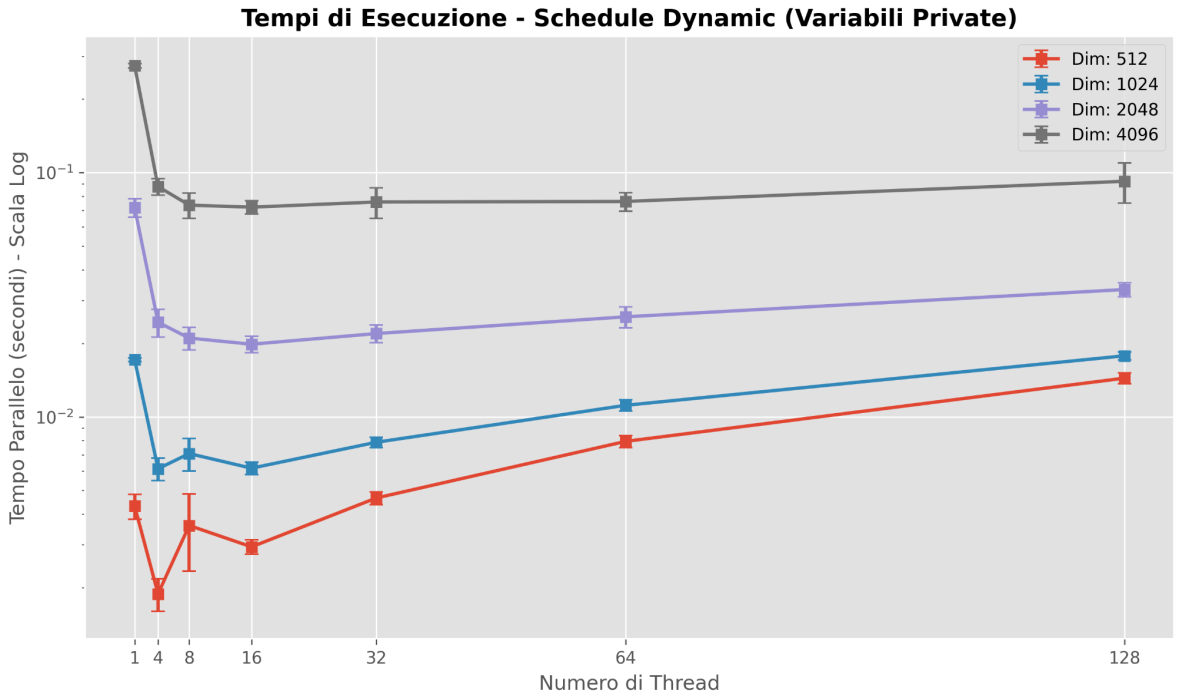
1.4 Metodologia di Profilazione

Per garantire il massimo rigore scientifico e la stabilità statistica dei risultati, il benchmark è stato automatizzato tramite uno script *Makefile* dedicato. Per ogni combinazione di parametri (dimensione della matrice N, numero di thread T, policy di scheduling e gestione della visibilità delle variabili):

1. Il tempo di esecuzione della singola regione parallela (escludendo le fasi sequenziali di allocazione e inizializzazione) è stato misurato utilizzando la libreria ad alta precisione *std::chrono::high_resolution_clock.*
2. L'esecuzione è stata ripetuta per **100 iterazioni**.
3. Il risultato finale utilizzato per la generazione dei grafici e il calcolo dello *Speedup* rappresenta la media aritmetica di queste 100 esecuzioni. Questo approccio è indispensabile per mitigare l'interferenza dovuta al "rumore di fondo" del sistema operativo (interrupt, scheduling di altri processi background) e per ammortizzare il costo di *cold-start* della cache della CPU nelle prime iterazioni.

2. Analisi della Scalabilità e Limiti Architetturali

2.1 Impatto della Dimensione del Problema (Legge di Amdahl)

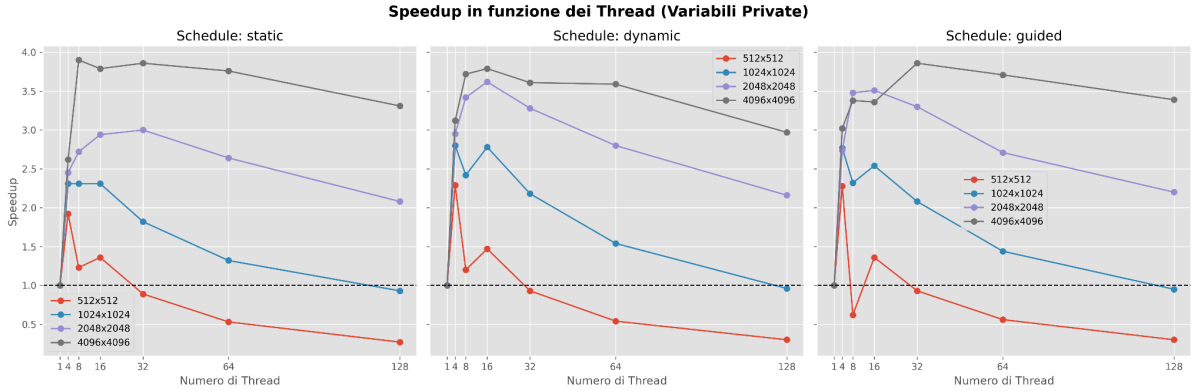


L'analisi dei risultati dimostra che il beneficio della parallelizzazione è strettamente correlato alla dimensione del problema, come previsto dalla **Legge di Amdahl**. Per matrici ridotte (es. N=512), il guadagno in termini di tempo di esecuzione è marginale. Al contrario, l'algoritmo manifesta un'ottima scalabilità su matrici di grandi dimensioni N=2048, N=4096).

Questo comportamento si spiega con la progressiva riduzione del peso relativo delle sezioni sequenziali (quali allocazione della memoria, inizializzazione e setup dei thread) rispetto al

carico computazionale parallelo man mano che la mole di dati aumenta. Tale dinamica permette allo Speedup di avvicinarsi in modo significativo al suo limite teorico.

2.2 L'Influenza dell'Architettura Hardware (Hyper-Threading)



L'andamento delle curve di Speedup rispecchia fedelmente l'architettura dell'Intel Core i7-4700MQ. Lo Speedup cresce quasi linearmente fino a 4 thread, in corrispondenza dei 4 core fisici, dove ogni thread può sfruttare un'unità aritmetico-logica (ALU) dedicata.

Superati i 4 thread, e fino a 8, la crescita dello Speedup prosegue ma con un ritmo sub-lineare. Questo è l'effetto della tecnologia **Simultaneous Multithreading (SMT), o Hyper-Threading**: i thread aggiuntivi condividono le risorse fisiche dei core (cache L1/L2, pipeline), mitigando le latenze di memoria, ma senza fornire un raddoppio effettivo della potenza di calcolo.

2.3 Limiti del Parallelismo e Oversubscription

Aumentando il grado di parallelismo a 16, 32, 64 e 128 thread, il sistema entra in uno stato di **oversubscription**. Il numero di thread logici supera drasticamente le 8 risorse hardware disponibili, costringendo il sistema operativo a un intenso **Context Switching** (prelazione e ripristino continuo dei thread).

Questo genera un notevole overhead dovuto al **Cache Thrashing** (invalidazione e ricaricamento incessante della cache L1). Tale inefficienza è aggravata dal fenomeno del **Memory Wall**: la banda passante della memoria RAM DDR3 non è sufficiente ad alimentare i core con la richiesta di dati. I thread saturano il bus di memoria, incorrendo in prolungati **memory stalls** durante il caricamento dei blocchi della matrice 4096x4096, rendendo l'ulteriore aggiunta di thread controproducente.

3. Analisi Approfondita delle Policy di Scheduling dei Thread in OpenMP

Al fine di comprendere l'influenza della strategia di distribuzione del carico di lavoro sulla performance e sull'efficienza del sistema, il kernel dell'algoritmo di convoluzione è stato sottoposto a una profilazione rigorosa. Questa analisi ha previsto l'impiego forzato delle tre policy di scheduling fondamentali messe a disposizione dalla specifica OpenMP: *static*, *dynamic* e *guided*. La scelta di queste direttive ha permesso di isolare l'impatto specifico di ciascuna metodologia di assegnazione del lavoro sul tempo di esecuzione complessivo e sui fenomeni di *overhead* a runtime.

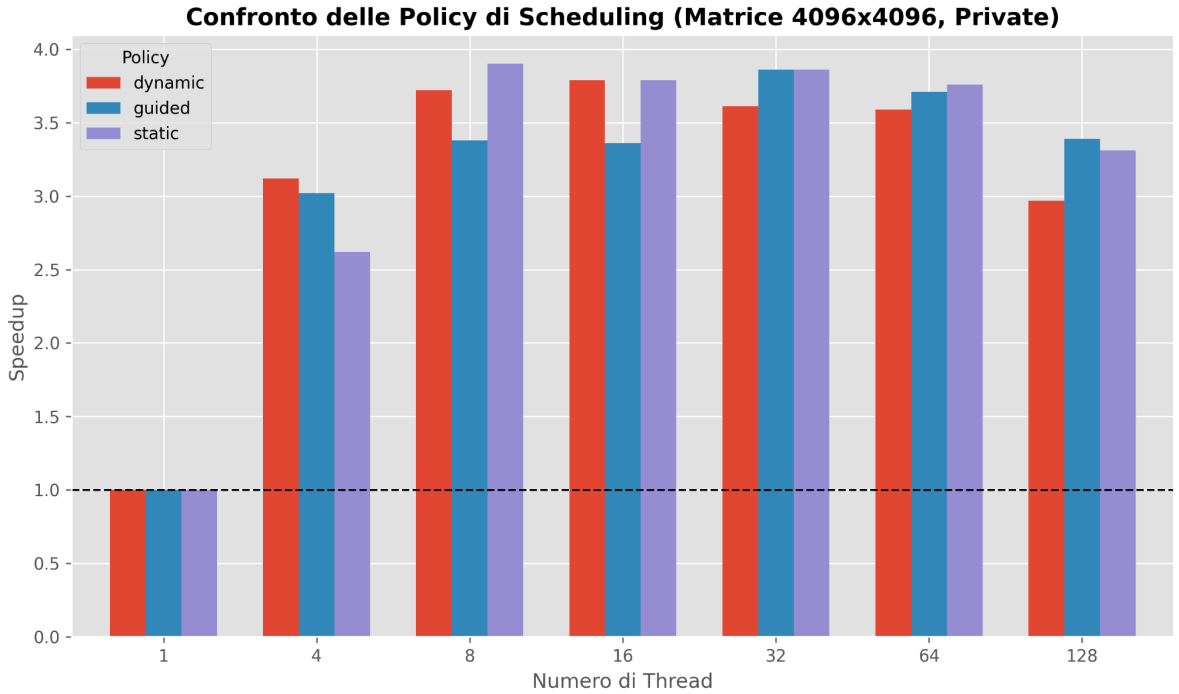
3.1 Caratteristiche del Carico di Lavoro: Regolarità, Bilanciamento e Uniformità Computazionale

L'implementazione di un filtro di convoluzione, in particolare con un *kernel* di dimensioni fisse come il 4x4, costituisce un archetipo di problema ad alta regolarità computazionale. La natura intrinsecamente omogenea di questo calcolo è cruciale per la scelta della policy di scheduling ottimale.

Punti chiave:

- **Perfetto Bilanciamento:** Ogni singola iterazione del ciclo esterno parallelizzato, che sia essa l'elaborazione di una riga o di un singolo pixel dell'immagine di output, esige un identico volume di operazioni aritmetiche. Nello specifico del filtro 4×4, ogni output richiede precisamente 16 moltiplicazioni e 16 somme in virgola mobile (operazioni FPU).
- **Assenza di Diramazioni:** La logica computazionale è strettamente sequenziale all'interno di ogni iterazione, priva di strutture condizionali complesse (if/else, switch) che possano generare percorsi di esecuzione di durata variabile. Questo elimina la possibilità di sbilanciamento dovuto a disuniformità nel codice (fenomeno noto come *computational imbalance*).
- **Granularità Fine e Costante:** La dimensione del lavoro (la convoluzione di un pixel) è relativamente *fine-grained* ma assolutamente costante.

3.2 Superiorità ed Efficienza della Policy di Scheduling Statico



I risultati sperimentali indicano in modo univoco che la policy *static* rappresenta la strategia più efficiente per questo specifico carico di lavoro. Il suo successo è legato alla minimizzazione quasi totale dell'*overhead* di coordinamento a runtime e alla sua capacità di massimizzare l'efficienza della memoria cache.

Meccanismo Operativo e Vantaggi:

1. **Zero Overhead a Runtime:** Con la direttiva schedule(static, chunk_size), il compilatore (o il runtime OpenMP) determina la divisione del lavoro *una tantum* in fase di inizializzazione del *team* di thread. Ad esempio, se ci sono N iterazioni e T thread, la dimensione del *chunk* è calcolata come N/T .

   L'assegnazione è fissata prima dell'inizio del ciclo, eliminando la necessità di qualsiasi sincronizzazione o comunicazione successiva tra i thread per l'ottenimento di nuovo lavoro. L'*overhead* di sincronizzazione per lo scheduling è quindi nullo.

2. **Massimizzazione della Località Spaziale:** L'assegnazione di blocchi contigui e consistenti di righe dell'immagine (o elementi della matrice) a un singolo thread massimizza l'efficacia del *caching*. Un thread che lavora su un ampio segmento contiguo di memoria genera una forte *località spaziale*, permettendo ai dati necessari di rimanere più a lungo e in modo più efficace nelle cache L1 e L2 del core CPU, riducendo drasticamente i *cache miss*.
3. **Mitigazione del False Sharing:** Assegnando grandi blocchi a ciascun thread, si riduce la probabilità di *false sharing*, una condizione in cui diversi thread modificano variabili non correlate che risiedono sulla stessa linea di cache, causando continui e costosi ricaricamenti.

3.3 Limiti e Costi delle Policy Dinamiche (*dynamic* e *guided*)

Nonostante le policy *dynamic* e *guided* siano indispensabili per la gestione ottimale di algoritmi intrinsecamente sbilanciati, esse si rivelano controproducenti e sub-ottimali in un contesto perfettamente bilanciato come la convoluzione.

Analisi dell'Overhead di Sincronizzazione:

1. **Sistema a Coda Centralizzata:** Entrambe le policy richiedono che i thread, una volta esaurito il loro piccolo *chunk* iniziale di lavoro, accedano a una *coda centrale di lavoro* gestita dal runtime OpenMP per "pescare" nuove iterazioni.
2. **Lock e Sezioni Critiche:** L'accesso a questa struttura dati condivisa (la coda) non può avvenire in modo concorrente e deve essere protetto tramite meccanismi di sincronizzazione espliciti o impliciti (tipicamente *lock* o *mutex*). L'acquisizione e il rilascio di questi lock per ogni richiesta di nuovo lavoro introduce un *micro-ritardo di latenza* non trascurabile.
3. **Impatto Negativo sul Fine-Grained:** Data l'estrema velocità di esecuzione del calcolo per un singolo pixel (poche decine di istruzioni FPU), il tempo speso per l'acquisizione del lock e l'aggiornamento della coda dello scheduler finisce per rappresentare una frazione significativa e negativa del tempo di esecuzione totale. L'overhead di sincronizzazione *supera il beneficio* del bilanciamento del carico, abbassando l'efficienza globale del sistema e aumentando il tempo di esecuzione complessivo rispetto alla policy static.

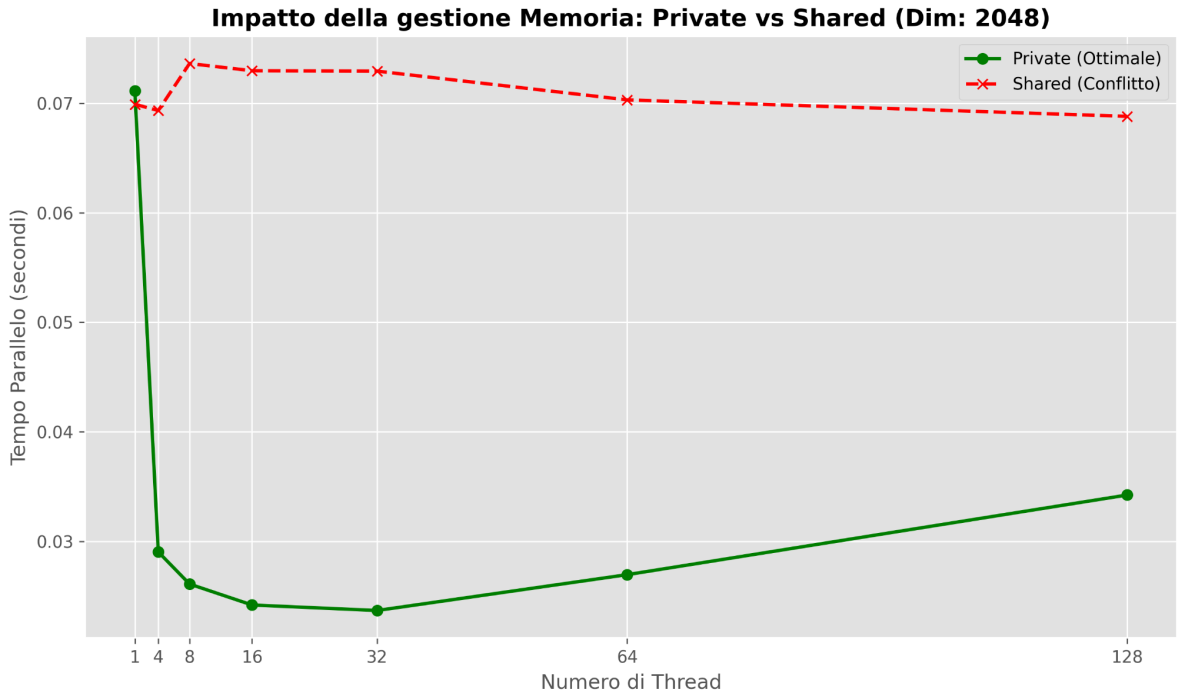
Differenza tra Dynamic e Guided:

- *dynamic*: Utilizza *chunk* di dimensione fissa, costante e spesso piccola.
- *guided*: La dimensione del *chunk* è inversamente proporzionale alla quantità di lavoro residuo. Inizia con *chunk* molto grandi e li riduce progressivamente fino a un valore minimo prefissato. Sebbene potenzialmente migliore del *dynamic* in scenari di sbilanciamento leggero, anche il *guided* introduce un overhead significativo a causa della logica di calcolo del *chunk size* e dell'accesso protetto alla coda.

4. Impatto della Gestione della Memoria: Variabili Private vs Shared nella Convoluzione Parallela

L'efficacia e la scalabilità di un algoritmo di elaborazione massiva come la convoluzione, quando implementato su architetture *Shared Memory* (sia UMA - Uniform Memory Access, sia NUMA - Non-Uniform Memory Access), non dipendono unicamente dalla corretta suddivisione del carico di lavoro. Un aspetto fondamentale, spesso trascurato, risiede nella profonda interazione tra il codice parallelo e la gerarchia della memoria fisica, in particolare il sistema di cache e i protocolli di coerenza.

Per analizzare empiricamente l'impatto della gestione esplicita della memoria, l'algoritmo di convoluzione, ottimizzato per il parallelismo *data-parallel* su OpenMP, è stato sottoposto a due distinti scenari di test, focalizzati sulla gestione della variabile di accumulo `somma`. Questa variabile, cruciale per raccogliere il risultato delle 16 moltiplicazioni e addizioni del kernel di 4x4 per ciascun pixel, rappresenta il collo di bottiglia potenziale per la contesa di accesso.



4.1 Lo Scenario Ottimale: Isolamento Massimale tramite Variabile Privata

Il primo scenario implementa il modello ideale per il parallelismo *data-parallel*, dove ogni thread opera in modo totalmente indipendente, senza necessità di comunicare o sincronizzare con gli altri thread durante la fase di calcolo. Questo è stato ottenuto dichiarando la variabile `somma` all'interno dello *scope* del doppio ciclo spaziale (riga e colonna) associato al singolo thread.

Vantaggi Architetturali e di Performance:

1. **Massima Prossimità Dati-Elaborazione:**
    - **Allocazione su Registri:** Il compilatore (in particolare con ottimizzazioni aggressive) riconosce immediatamente che la variabile `somma` è temporanea e strettamente locale. Di conseguenza, nella maggior parte dei casi, la variabile viene allocata direttamente nei registri fisici della CPU, che costituiscono il livello più alto e rapido della gerarchia di memoria.
    - **Allocazione su Stack/Cache L1:** In alternativa, la variabile risiede nello **Stack del thread**, che per sua natura è confinato alla memoria *Cache L1*

privata di quel core. Nel processore Intel i7 utilizzato, la cache L1 è *per-core*, garantendo l'isolamento fisico.

2. **Assenza Totale di Contesa (*Zero Contention*):**
    - Le operazioni di *Read-Modify-Write* (lettura del valore, addizione del nuovo prodotto, riscrittura) sulla variabile `somma` avvengono senza mai coinvolgere il bus di sistema o i protocolli di coerenza di cache. L'hardware può eseguire questa sequenza alla massima frequenza di clock consentita, garantendo il massimo ***throughput*** teorico per il calcolo. Non esiste il rischio di ***Race Conditions*** o la necessità di ***Atomic Operations***.
    - **Modello Mentale:** Questo approccio riflette fedelmente il modello di programmazione "divide et impera", dove l'assenza di dipendenze di memoria tra le unità di calcolo (i thread) garantisce l'assoluta scalabilità.

4.2 Lo Scenario di Conflitto: Il Costo della Sincronizzazione e il False Sharing

Il secondo scenario è stato concepito per dimostrare la penalità associata a un uso improprio o *naïve* della memoria condivisa, forzando la variabile `somma` ad essere accessibile (o visibile) a tutti i thread.

Problematiche Algoritmiche: La *Race Condition*

In una prima implementazione non protetta, la condivisione della variabile temporanea di accumulo conduce inevitabilmente a ***Race Conditions***. Due o più thread tentano di aggiornare simultaneamente la medesima locazione di memoria: l'operazione di uno sovrascrive il risultato parziale dell'altro, portando a un ***output visivo corrotto*** e a risultati numerici inaffidabili.

Soluzione di Correttezza e Implicazioni di Performance:

Per garantire la correttezza algoritmica, l'accesso alla variabile `somma` condivisa deve essere gestito da ***primitive di sincronizzazione***. L'utilizzo della direttiva OpenMP `#pragma omp atomic` (o equivalenti *lock*) è il meccanismo standard per assicurare l'accesso mutuamente esclusivo (*Mutual Exclusion*).

Tuttavia, l'impiego di atomiche sul calcolo di *ogni singolo pixel* introduce colli di bottiglia catastrofici che annullano i benefici del parallelismo:

1. **Serializzazione (*Funnel Effect*):** Il costrutto atomico costringe i thread a operare ***in sequenza*** sulla risorsa condivisa. Di fatto, la sezione critica viene serializzata, trasformando la porzione di codice parallelo in un'esecuzione essenzialmente sequenziale. Il ***fattore di speedup*** tende rapidamente all'unità, indipendentemente dal numero di core utilizzati.
2. **Cache Thrashing e Protocollo MESI (*False Sharing*):**
    - Anche in un caso leggermente meno grave, dove i thread operassero su variabili condivise *adiacenti* (ma non identiche) in memoria, se queste risiedono nella medesima ***Cache Line*** (tipicamente 64 byte), si innesca il

fenomeno del ***False Sharing***.

- I core fisici aderiscono rigorosamente ai ***Protocolli di Coerenza di Cache*** (il più comune è il ***MESI*** - *Modified, Exclusive, Shared, Invalid*). Se un core modifica un singolo byte all'interno di una Cache Line (stato *Modified*), gli altri core che hanno una copia di quella Cache Line nelle loro L1 la vedono immediatamente invalidata (stato *Invalid*).
- Questo innesca un ciclo vizioso (*Cache Thrashing*): i core sono costretti a ***invalidare e ricaricare continuamente*** (o *fetch*) la linea di cache dal livello successivo e più lento (Cache L3 o RAM principale). Il tempo speso in comunicazione sul bus di sistema e ricaricamenti di memoria supera di gran lunga il tempo effettivo di calcolo.

4.3 Il Ruolo Cruciale del Compilatore: Data-Flow Analysis e *Register Promotion*

Osservando i risultati del benchmark, emerge un fenomeno apparentemente controintuitivo: i tempi di esecuzione per la versione "Private" e quella "Shared" (nonostante la necessità teorica di atomiche) risultano ***quasi coincidenti***. Non si è osservato il degrado verticale delle prestazioni previsto nello scenario di contesa.

Questo risultato non è sintomo di efficienza della memoria condivisa, ma la prova diretta di ***ottimizzazioni aggressive*** da parte del compilatore *g++* (attivate dal flag *-O3*).

Meccanismi di Ottimizzazione:

1. **Data-Flow Analysis (Analisi del Flusso Dati):** Il compilatore esegue un'analisi statica complessa. Identifica che, sebbene la variabile *somma* sia stata *definita* a livello globale o condiviso dallo sviluppatore, essa viene reinizializzata a ***0.0f*** all'inizio del calcolo di ogni nuovo pixel. Crucialmente, non esiste alcuna dipendenza di dati (lettura o scrittura) tra le iterazioni del ciclo (i pixel) o con altri thread.

2. **Privatization e *Register Promotion*:** Riconoscendo l'assenza di dipendenze, il compilatore ignora silenziosamente la direttiva (imperfetta) dello sviluppatore e attua due tecniche fondamentali:
    - **Privatization:** La variabile condivisa viene automaticamente *privatizzata* per lo scope del thread.
    - ***Register Promotion:*** La variabile viene promossa a ***registro locale*** del core.

Il compilatore ha agito come una ***"rete di sicurezza"***, trasformando la versione concettualmente errata "Shared" in una versione internamente ottimizzata "Private", evitando così l'accesso alla memoria primaria e le penalità di *False Sharing*.

4.4 Considerazioni di Design Software e Portabilità

Nonostante l'efficacia delle ottimizzazioni offerte dai compilatori moderni, a livello di *Software Design* la dipendenza acritica da queste reti di sicurezza è unanimemente considerata una ***bad practice***. Il codice sorgente dovrebbe riflettere esplicitamente l'intento di parallelismo e la gestione della memoria.

Principi Essenziali per un Design Robusto:

1. **Garanzia di Portabilità e Performance Consistente:**
    - **Compilatori Diversi:** Compilatori differenti (es. Intel ICC, Microsoft Visual Studio C++), o anche versioni diverse dello stesso compilatore, potrebbero non eseguire la stessa *Data-Flow Analysis* o la stessa *Register Promotion*.
    - **Livelli di Ottimizzazione:** Codice compilato a bassi livelli di ottimizzazione (es. `-O0` per il *debug*) non attiverà queste euristiche. La migrazione su architetture diverse o la modifica del *build system* potrebbe inaspettatamente riportare alla luce i problemi di contesa, portando a ***crollo di performance inspiegabili***.
2. **Correttezza Semantica e Trasparenza:**
    - Dichiarare esplicitamente le variabili temporanee con lo *scope* corretto (variabili locali all'interno dei cicli, o tramite clausole OpenMP come *private(...)*) è l'unico approccio che ***garantisce per design*** l'assoluta assenza di conflitti di memoria.
    - Questa pratica mantiene il codice leggibile e il modello mentale del parallelismo (l'indipendenza delle operazioni) trasparente allo sviluppatore.

5. Conclusioni

L'analisi empirica condotta sull'algoritmo di convoluzione 2D, parallelizzato tramite direttive OpenMP, ha confermato la natura intrinsecamente data-parallel del problema. Tuttavia, lo studio ha ampiamente dimostrato come l'ottenimento di un elevato Speedup non sia una diretta conseguenza dell'aggiunta indiscriminata di thread logici, bensì il risultato di un delicato bilanciamento tra l'algoritmo software e l'architettura hardware sottostante.

In primo luogo, in perfetto accordo con la **Legge di Amdahl**, si è osservato che l'efficienza scala positivamente all'aumentare del carico di lavoro (la dimensione della matrice), riducendo l'impatto percentuale delle frazioni sequenziali del programma. Nonostante ciò, le prestazioni si scontrano con due limiti fisici invalicabili:

1. **Oversubscription:** Spingere il numero di thread oltre le risorse di esecuzione fisiche e logiche della CPU (es. oltre gli 8 thread) genera un overhead di context-switching che degrada attivamente le prestazioni.
2. **Memory Wall:** L'elevata capacità di calcolo dei core satura rapidamente la limitata banda passante della memoria RAM, portando i processori in uno stato di stallo (memory stalls) in attesa del caricamento dei dati.

In secondo luogo, dal punto di vista della distribuzione del carico, la topologia regolare della matrice elegge inequivocabilmente lo **scheduling** *static* come policy ottimale. Suddividere il lavoro in chunk fissi e contigui a tempo di avvio annulla l'overhead di sincronizzazione a runtime (inevitabile invece con le code dinamiche delle policy *dynamic* e *guided*) e massimizza la località spaziale dei dati pre-caricati nella cache L1/L2.

Infine, l'indagine ha rimarcato la criticità della **gestione della coerenza della memoria**. La profilazione ha dimostrato che la contesa per l'accesso a variabili temporanee condivise innesca letali colli di bottiglia legati ai protocolli di Cache Coherence e al fenomeno del False Sharing. Sebbene in fase di ottimizzazione aggressiva (`-O3`) il compilatore attui tecniche salvavita di Register Promotion privatizzando autonomamente tali variabili, la progettazione di un software parallelo robusto impone l'uso esplicito del corretto scoping locale. Solo forzando la **stretta località dei dati privati** si garantisce l'assoluta assenza di race conditions silenti, evitando serializzazioni involontarie sul bus di sistema a prescindere dall'ambiente di compilazione.

In sintesi, la massimizzazione delle prestazioni nell'High Performance Computing su architetture a memoria condivisa richiede un profondo co-design: non basta parallelizzare il calcolo, ma è fondamentale orchestrare magistralmente la gerarchia della memoria e le logiche di sincronizzazione.